Dimension B: Rule-Variety Cost — The Rule-Authoring Cost Dimension as Standalone Architectural Treatment in CKS

Author: Wenxin Li (Independent Researcher)

ORCID: 0009-0004-8065-3235

Date: May 4, 2026

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in 'Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems' (Li, April 2026). It does not introduce new axioms. Its sole contribution is to formalize Dimension B — rule-variety cost — as a standalone architectural treatment within the linear-cost scaling decomposition that parent foundational note A1.06 commits to.

Abstract

CKS commits to linear-cost scaling as one of its six architectural commitments. The integrating-frame note A2.29 decomposes this commitment into four cost dimensions, of which Dimension B treats the cost driven by rule variety. This note formalizes Dimension B as a standalone architectural treatment. It articulates what rule variety measures (R-count and R-complexity, both deployment design choices), what cost behavior CKS commits to on this dimension (rule-authoring cost grows with R at design time and is amortized across cell executions; rule maintenance cost grows with R; per-execution rule-application cost is bounded by cell scope per A2.09 rather than by R), distinguishes Dimension B's amortization property from four adjacent cost patterns commonly conflated with it, names eight failure modes in which implementations break amortization, and provides an operational test for whether a deployment respects the Dimension B commitment.

1. Why Dimension B needs to be formalized as standalone

The parent note A1.06 commits CKS to linear-cost scaling — infrastructure cost that is database-like-additive across the relevant cost dimensions rather than superlinear in the way fine-tuning, continual learning, or multi-agent coordination overheads are (§6). The integrating-frame note A2.29 decomposes that commitment into four dimensions: substrate size (Dimension A, treated standalone in A2.30), rule variety (Dimension B, this note), intervention frequency (Dimension C, A2.32), and per-cell workload (treated standalone in A2.33). A fifth standalone treatment, A2.34, formalizes the size-independence-of-governance property as the cross-dimensional invariant the four together produce.

Standalone treatment of Dimension B is needed for three reasons.

First, deployments need to assess how their costs respond as their rule sets evolve. CKS deployments typically begin with a small set of orchestration rules covering core cell-execution patterns and expand the set as new situations arise. Without a precise specification of what cost behavior the architecture commits to on the rule-variety dimension, deployments cannot plan rule-set evolution, and downstream implementers cannot defensibly claim CKS-coherence on this axis.

Second, the amortization property of rule authoring — rule cost paid once at design time, applied many times across cell executions — is a consequential prior-art claim. Patentable derivations focused on rule-authoring frameworks, rule-application engines, or rule-evaluation patterns are substantially more defensibly contested when Dimension B's amortization property is formalized as standalone public prior art rather than as a sub-clause of the broader linear-cost commitment.

Third, Dimension B pairs directly with the rule-authoring moment formalized in A2.04. A2.04 establishes rule authoring as a governance moment exercised by humans at design time; this note specifies the cost behavior of that moment. The two notes are conjugate.

2. What Dimension B measures and how it grows

Rule variety, as a cost driver, has two operational components.

R-count. The number of distinct orchestration rules in the deployment's substrate. Each rule is itself substrate content per A2.04 — addressable, inspectable, modifiable, and overridable under human governance per A1.01. R-count grows when humans author new rules to cover new situations, refine existing rules through replacement, or expand rule coverage to new domains. R-count shrinks when humans deprecate rules that no longer apply.

R-complexity. The complexity of individual rules. A simple rule (e.g., "when content of type X is written, also record field Y") has low complexity; a complex rule (e.g., "when content of type X is written under condition Z, with constraint Q, record field Y with rationale derived from substrate state W") has high complexity. R-complexity contributes both to per-rule authoring cost and to per-rule maintenance cost.

Total rule-variety cost is approximately R-count × average R-complexity per rule. A deployment with one hundred simple rules may have similar total cost to a deployment with ten complex rules, depending on how complexity is distributed. The architecture supports both patterns; deployments choose the distribution that matches their domain and governance pattern.

R grows in three patterns deployments commonly exhibit. *Coverage expansion:* new rules are authored to address situations existing rules do not cover, increasing R-count. *Rule refinement:* existing rules are revised in response to observed misfires or inadequacies, often increasing R-complexity. *Rule deprecation:* rules that no longer apply are removed, decreasing R-count. The architectural commitment is that R can grow or shrink as deployments evolve, and the cost properties Dimension B commits to hold across the range.

3. The cost behavior of Dimension B

The architectural commitment on Dimension B has three operational components.

(a) Rule-authoring cost grows with R, paid at design time. Authoring rule N+1 requires the same architectural effort as authoring rule N (modulo per-rule complexity differences); a deployment with one hundred rules has paid roughly one hundred times the rule-authoring cost of a deployment with one rule. This is straightforward design-time scaling on the R dimension, paid at the rule-authoring moment per A2.04.

(b) Rule-application cost is amortized across cell executions. Once a rule has been committed to substrate per A2.04, it applies to every cell execution within its scope. The per-rule cost is paid at authoring time; the per-execution cost of applying the rule is bounded by the cell's scope per A2.09 and is independent of R. A cell that operates under five rules does not have five times the per-execution cost of a cell that operates under one rule, provided the cell's scope is bounded.

(c) Rule maintenance cost grows with R. Rules require occasional review, refinement, and deprecation as deployments evolve. Maintenance cost is roughly proportional to R, with deployments operating at higher R levels paying proportionally more maintenance attention than deployments at lower R levels. Maintenance cost is paid at maintenance time, not at execution time.

The three components together specify Dimension B's cost behavior. The architectural commitment is to design-time payment with execution-time amortization, not to per-execution payment; deployments may trade R-count and R-complexity against each other based on their governance pattern, and the architecture is neutral on which trade is preferred.

4. What the Dimension B commitment does NOT claim

Five clarifications are worth stating to keep the standalone treatment from being overstated.

It does not claim that rule authoring is cheap in absolute terms. Authoring a single complex rule may require substantial human effort. The architectural commitment is to cost behavior — where the cost is paid and what it is paid relative to — not to absolute levels.

It does not constrain how R grows. Deployments choose their rule sets; the architecture does not specify minimum or maximum R or any optimal R-count target.

It does not claim that all rules are equally costly. Rule complexity varies widely; a simple rule may be authored in minutes while a complex rule may take hours or days. The architecture supports any complexity distribution; it does not dictate one.

It does not claim a specific authoring time per rule. Authoring time depends on the rule's complexity, the human author's expertise, and the deployment's rule-authoring tooling; the architecture is technology-agnostic on these factors per A1.05.

It does not claim that rule re-authoring is free. When deployments revise rules, the cost of re-authoring is paid per revision. Frequent revision of complex rules can produce substantial cumulative cost, but the architecture supports this pattern; rule modification is itself a governance moment per A2.04.

5. What Dimension B is NOT — four adjacent cost patterns

Four adjacent cost patterns are commonly conflated with Dimension B. The amortization property forbids each conflation.

Not per-execution rule evaluation cost. Some implementations pay rule-evaluation cost at every cell execution, with evaluation cost growing with R as cells consult more rules during execution. Dimension B's amortization property forbids this conflation: rule cost is paid at authoring time, not at execution time. Cell execution may consult rules, but the consultation is bounded by the cell's scope per A2.09, not by R.

Not rule-validation cost applied per write. Some implementations validate substrate writes against all rules before committing, with validation cost growing with R per write. This is per-execution cost masquerading as governance; Dimension B's amortization property says rule cost is paid at authoring time, not at write time. Validation that a write satisfies the rules a cell operates under is bounded by the cell's scope, not by R.

Not rule-discovery and rule-search cost. Some implementations require cells to discover applicable rules at execution time, searching through the rule set for matches. Discovery cost grows with R as per-execution cost. The architectural commitment is that rules are addressable per A2.04 and cells operate under specific rules per A2.20; discovery should be design-time, not execution-time.

Not ML-policy training cost. Some implementations train ML policies that learn from substrate content and produce policy-driven decisions. ML training has its own cost dimensions — training-data size, model complexity, retraining frequency — that do not map to Dimension B. Per A2.20, ML-trained policies are not orchestration rules in the architectural sense; their cost properties are deployment concerns separate from the rule-cost dimension.

6. Why Dimension B is load-bearing for downstream commitments

Dimension B's amortization property is load-bearing for several other CKS commitments.

For the rule-authoring moment per A2.04. The architectural commitment to rule authoring as design-time governance depends on Dimension B's amortization property — rules are authored once and applied many times. Without amortization, rule authoring would be effectively per-execution and would not be a distinct governance moment.

For the two-moment governance structure per A2.04 and A2.05. The two governance moments — rule authoring (Moment 1, A2.04) and direct override (Moment 2, A2.05) — are cost-bearing on R (Dimension B) and F (Dimension C, A2.32) respectively. Their distinction depends on rule cost being amortized at design time. Without Dimension B's amortization, rule authoring and direct override would collapse into a single per-execution governance pattern.

For the size-independence guarantee on Dimension A per A2.30. Governance cost not growing with N is operationalizable only because rule cost is paid on R rather than on N. If rule cost were per-substrate-element (paid for each substrate element a rule applies to), governance cost would scale

with N rather than with R, and the Dimension A guarantee would not hold.

For Property B of the AI-as-substrate-mediator commitment per A2.20 (LLM writes under orchestration rules). The mediator commitment depends on rules applicable to LLM writes at execution time without per-execution authoring cost; Dimension B's amortization is what makes this feasible at any non-trivial R.

For the labor allocation framework per A1.12. Mode 2 (LLM under rule) is cost-effective only because rule authoring is amortized across the cells that operate under each rule. Without amortization, LLM-under-rule labor would be more expensive than direct human labor for any non-trivial deployment, and the three-mode allocation framework would collapse.

7. Failure modes that violate Dimension B's amortization

Each anti-pattern below names a way an implementation can produce rule-cost behavior that scales as per-execution rather than design-time amortized.

Per-execution rule evaluation. The implementation evaluates the entire rule set at every cell execution, with evaluation cost growing with R per execution. Amortization fails; rule cost behaves as if paid per execution.

Rule-discovery at execution time. Cells discover applicable rules during execution by searching the rule set, with discovery cost growing with R per execution. The discovery cost is functionally per-execution rule cost, even when the implementation calls it indexing or matching.

Rule-validation per write. Substrate writes are validated against all rules before committing, with validation cost growing with R per write. Rules become per-write cost rather than per-rule cost.

Rule re-evaluation on substrate change. Whenever substrate content changes, all rules are re-evaluated against the new state, with re-evaluation cost growing with R per change. Rules become continuously paid rather than design-time paid.

Rule-cost amortization broken by indexing. The implementation maintains rule indexes that grow with R, with index-maintenance cost paid per write or per substrate change. The indexing cost behaves as per-execution rule cost even though it is technically indexing infrastructure.

Rule-conflict resolution at execution time. The implementation detects conflicts among rules at execution time, with conflict-detection cost growing with R-squared per execution. Conflict detection should be design-time — A2.04 rule authoring requires humans to consider rule interactions when rules are authored — not execution-time.

Rule-explosion through composition. The implementation allows rules to compose dynamically, with the effective rule count at execution time exceeding the authored R. Per-execution evaluation grows with the composed count rather than with the authored count.

Rule-application requires LLM context expansion with R. The implementation passes all R rules to the LLM mediator at execution time, with LLM context size growing with R. LLM cost (token cost,

latency) becomes per-execution growing with R, even when only a subset of rules is relevant to the specific cell execution. This is a particularly consequential failure mode in current LLM-mediator implementations because it appears to satisfy A2.20's "LLM writes under orchestration rules" commitment while silently breaking the amortization property.

8. Operational test

A deployment respects Dimension B's amortization property if and only if all of the following are true at all times during the deployment's existence:

1. Rule-authoring cost is paid per rule at the rule-authoring moment per A2.04, not per cell execution.
2. Rule-application cost during cell execution is bounded by the cell's scope per A2.09, not by R; cells operating under more rules do not have proportionally higher per-execution cost.
3. The rules a cell operates under are determined at design time (or at the cell's instantiation time), not discovered at execution time.
4. Rule changes are paid per modification at the rule-authoring moment, not per cell execution that occurs after the modification.
5. Rule maintenance cost — review, refinement, deprecation — is paid at maintenance time, not amortized inappropriately into per-execution costs.

A deployment that fails any of (1)–(5) does not respect Dimension B's amortization property in the architectural sense, even if its absolute rule-cost is acceptable in practice.

9. Why naming Dimension B as standalone matters

Implementations under pressure to support flexible rule patterns, dynamic rule application, or sophisticated rule-discovery features consistently drift toward per-execution rule cost. The drift is steady because per-execution evaluation feels architecturally cleaner than design-time scoping, and modern rule engines often default to per-execution patterns. The drift is also steady because LLM-mediator implementations face a recurring temptation to pass the full rule set into the model's context window — preserving the appearance of A2.20's commitment while breaking the amortization property silently.

Implementations that drift away from Dimension B's amortization produce systems that work at small R but become operationally infeasible as rule sets grow. Per-execution evaluation cost grows with R; cells take longer to execute as rules accumulate; LLM context costs balloon as more rules are passed at execution time; index-maintenance overheads consume the savings the substrate was supposed to deliver. Each crisis traces back to rule cost that was allowed to scale per-execution when the architecture committed to design-time amortization.

Naming Dimension B as a standalone architectural commitment gives downstream implementers a precise specification of what cost behavior CKS commits to on the rule-variety dimension. The subsequent notes A2.32, A2.33, and A2.34 specialize the remaining dimensions and the size-independence-of-governance property; together they give the full decomposition of A1.06.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Dimension B: Rule-Variety Cost — The Rule-Authoring Cost Dimension as Standalone Architectural Treatment in CKS.* May 4, 2026. ORCID: 0009-0004-8065-3235.